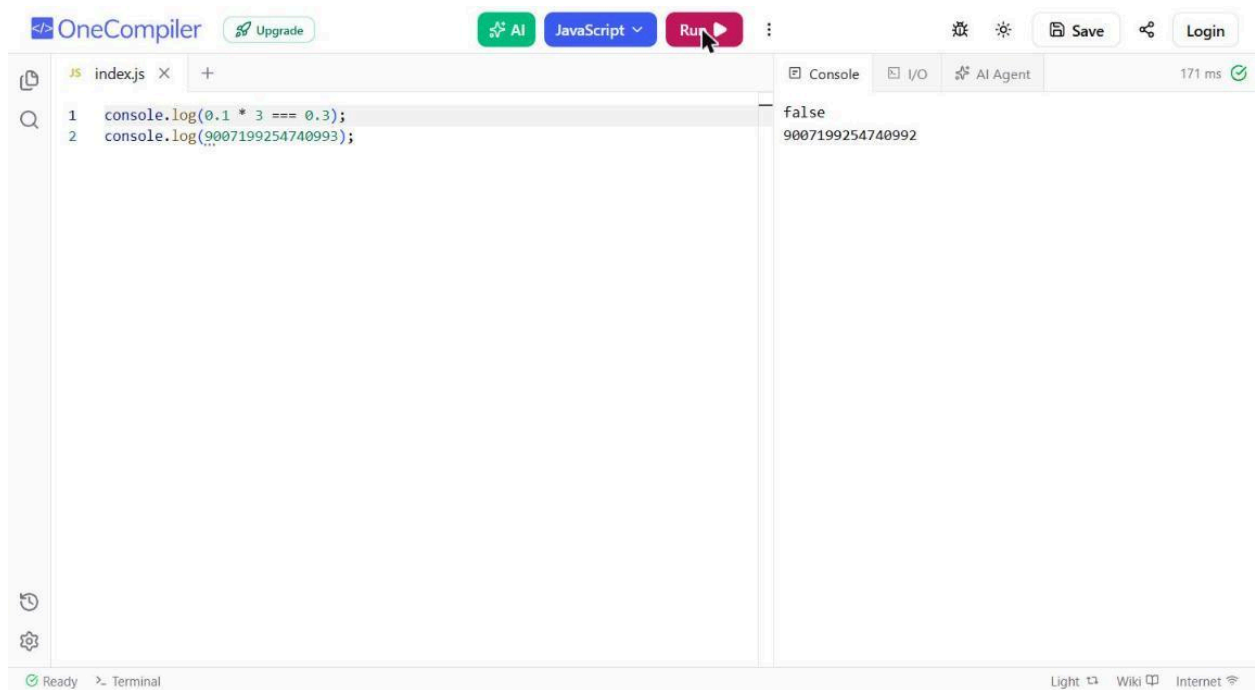


# Exercício em duplas — Preveja antes de executar

Eduardo Barella - 24088000-2

Matheus Mascarello - 24120740-2

## 1 · JavaScript



The screenshot shows the OneCompiler web interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'JavaScript', and 'Run'. The 'Run' button is highlighted with a mouse cursor. Below the top bar, there are tabs for 'Console', 'I/O', and 'AI Agent'. The 'Console' tab is active, showing the output of the JavaScript code. The code in the editor consists of two lines: `console.log(0.1 * 3 === 0.3);` and `console.log(9007199254740992);`. The console output shows `false` for the first line and `9007199254740992` for the second line. The status bar at the bottom indicates 'Ready' and 'Terminal'.

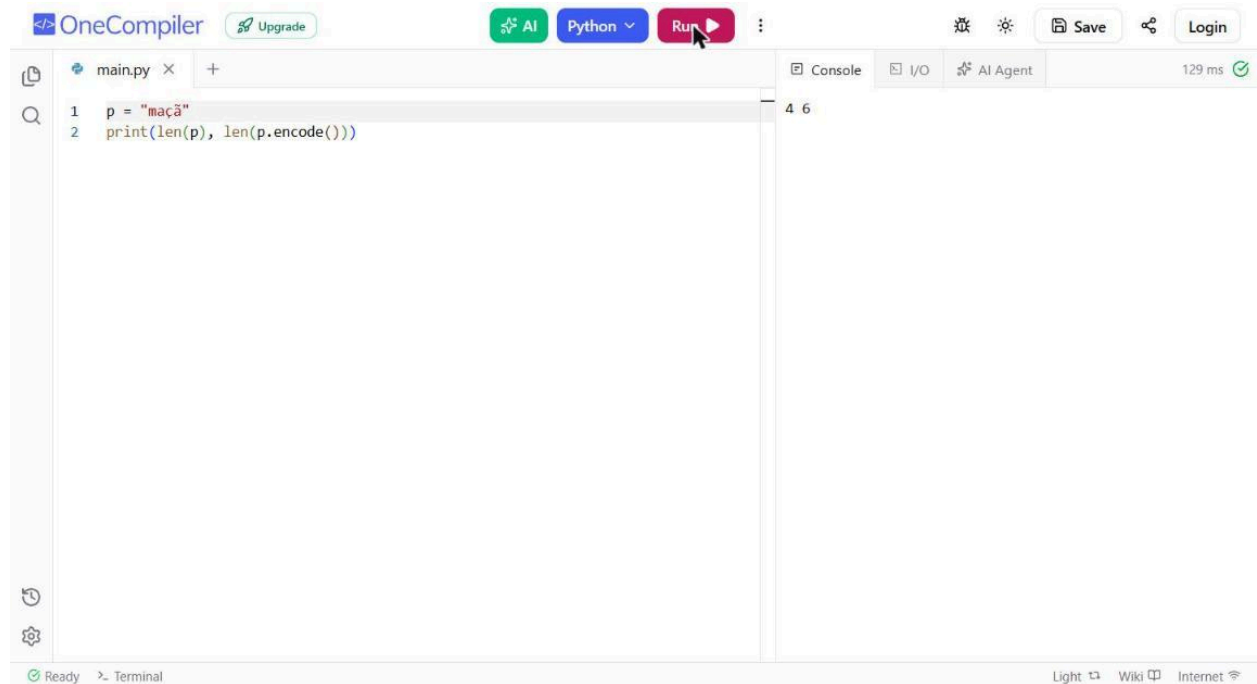
```
1 console.log(0.1 * 3 === 0.3);
2 console.log(9007199254740992);
```

Output: false  
9007199254740992

(a) Saída: false e 9007199254740992

(b) Nunca.  $0.1 \times 3$  vira 0.30000000000000004 por causa do ponto flutuante binário. O número passa do `Number.MAX_SAFE_INTEGER` ( $2^{53}-1$ ) e é arredondado sem aviso.

## 2 · Python



The screenshot shows the OneCompiler web interface. At the top, there's a navigation bar with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', 'Save', and 'Login'. Below the navigation bar, the main editor area displays a file named 'main.py' with the following code:

```
1 p = "maçã"
2 print(len(p), len(p.encode()))
```

To the right of the code editor, the output of the program is shown as '4 6'. The bottom status bar indicates 'Ready' and 'Terminal'.

(a) Saída: 4 6

(b) Nunca. `len(p)` conta caracteres (4). `encode()` gera bytes em UTF-8, e "ç" e "ã" ocupam 2 bytes cada:  $1+1+2+2 = 6$ .

### 3 - Go

The screenshot shows the OneCompiler web interface with a Go file named 'main.go'. The code defines a 'main' package with a 'main' function. Inside the function, a variable 'b' of type 'byte' is initialized to 255, incremented by 1, and then printed using 'fmt.Println(b)'. The console output shows '0'. The top navigation bar includes buttons for AI, Go, Run, Save, and Login. The bottom status bar indicates 'Ready' and '259 ms'.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var b byte = 255
7     b++
8     fmt.Println(b)
9 }
```

0

Ready 259 ms

(a) Saída: 0

(b) Nunca. byte é uint8 (0 a 255), e o b++ estoura e volta pra 0 sem erro.

## 4 · Java

The screenshot shows the OneCompiler web interface with a Java file named 'Main.java'. The code defines a 'Main' class with a 'main' method. Inside the method, an array 'v' of type 'int' is initialized with 3 elements. The first element 'v[0]' is printed, and then 'v[3]' is printed, which causes an 'ArrayIndexOutOfBoundsException'. The console output shows the exception message. The top navigation bar includes buttons for AI, Java, Run, Save, and Login. The bottom status bar indicates 'Ready' and 'Terminal'.

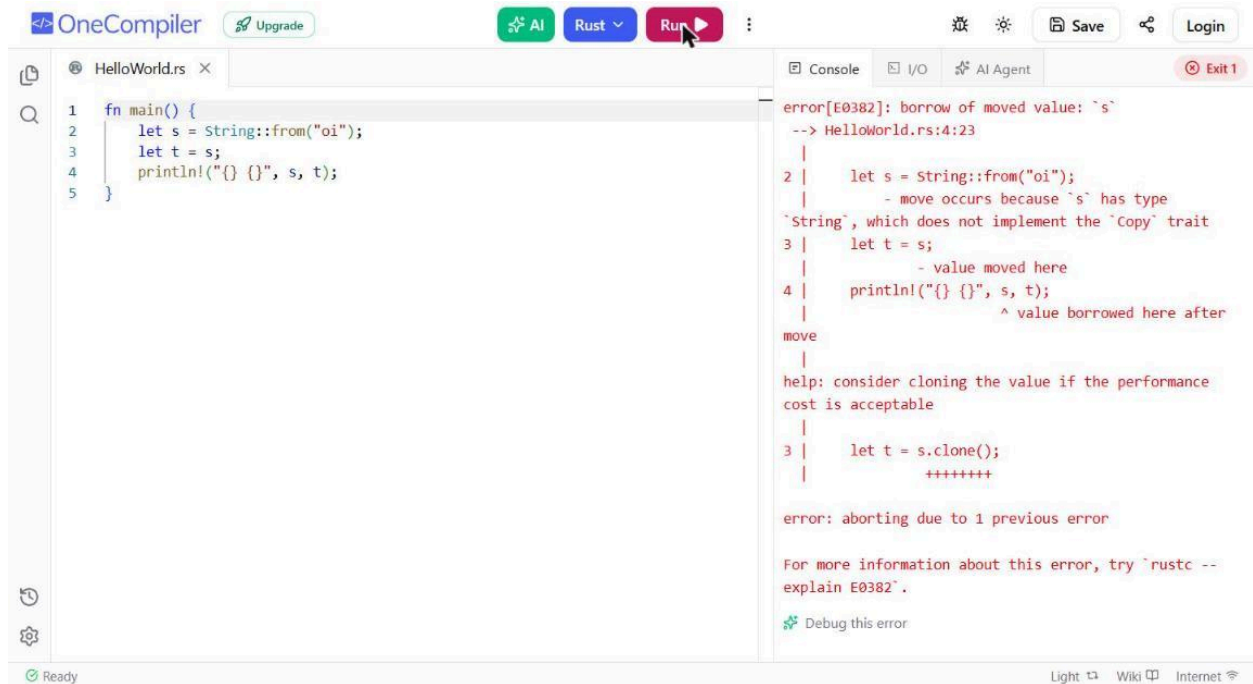
```
1 public class Main {
2     public static void main(String[] args) {
3         int[] v = new int[3];
4         System.out.println(v[0]);
5         System.out.println(v[3]);
6     }
7 }
```

Exception in thread "main"  
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3  
at Main.main(Main.java:5)

Ready Terminal

- (a) Saída: 0, depois `ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3`
- (b) Execução. Arrays de `int` começam zerados e os índices válidos são 0 a 2. A JVM verifica o limite em tempo de execução.

## 5 · Rust



The screenshot shows the OneCompiler web IDE with a Rust file named `HelloWorld.rs`. The code in the editor is:

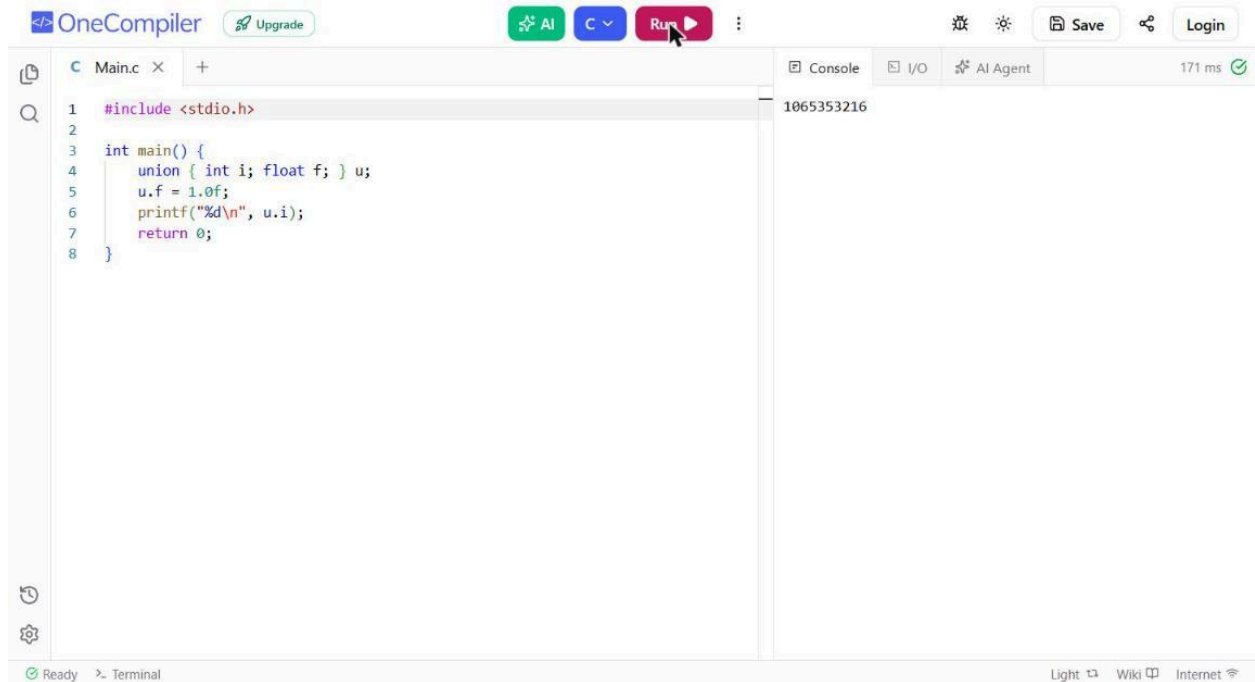
```
1 fn main() {  
2     let s = String::from("oi");  
3     let t = s;  
4     println!("{}", s, t);  
5 }
```

The right-hand pane displays the compiler error:

```
error[E0382]: borrow of moved value: `s`  
--> HelloWorld.rs:4:23  
|  
2 |     let s = String::from("oi");  
|     - move occurs because `s` has type  
|     `String`, which does not implement the `Copy` trait  
3 |     let t = s;  
|         - value moved here  
4 |     println!("{}", s, t);  
|                   ^ value borrowed here after  
| move  
|  
| help: consider cloning the value if the performance  
| cost is acceptable  
|  
3 |     let t = s.clone();  
|         +++++++  
|  
error: aborting due to 1 previous error  
  
For more information about this error, try `rustc --  
explain E0382`.  
  
Debug this error
```

- (a) Saída: nenhuma, porque o programa não compila
- (b) Compilação (erro E0382). "let t = s" move a posse da `String` para `t`, e o borrow checker impede usar `s` depois disso.

## 6 · C



The screenshot shows the OneCompiler web IDE. The code editor contains the following C code:

```
1 #include <stdio.h>
2
3 int main() {
4     union { int i; float f; } u;
5     u.f = 1.0f;
6     printf("%d\n", u.i);
7     return 0;
8 }
```

The console output on the right shows the result of the program execution: 1065353216. The execution time is 171 ms.

(a) Saída: 1065353216

(b) Nunca. A union guarda int e float na mesma memória. Os bits de 1.0f (IEEE 754 = 0x3F800000) lidos como int dão 1065353216.

## Conclusão

compilação → Rust | execução → Java | nunca → JavaScript, Python, Go e C